

School Of Future Tech

Case Study Report On:

Smart Electric Vehicle Charging Network

Dynamic Station Management Using Singly Linked List

By

Ashutosh Pawar

150096725130

B.Tech CSE 2025–29 | Semester II
ITM Skills University, Kharghar, Navi Mumbai
Data Structures & Algorithms with C++

June 2026

Table of Contents

1. Introduction to the Case Study
2. Problem Statement / Case Background (Abstract)
3. System Design & Architecture
4. Methods, Algorithms & Technology Applied
5. Implementation Details
6. Results and Conclusion
7. References

1. Introduction to the Case Study

The global transition toward electric vehicles (EVs) has fundamentally changed how cities and organizations think about transportation infrastructure. As of 2024, over 40 million electric vehicles are on roads worldwide, a number projected to exceed 300 million by 2030. This rapid growth creates an urgent need for intelligent, dynamic, and scalable charging infrastructure.

Unlike traditional fuel stations, EV charging stations require real-time status management — a single station can shift between Available, Occupied, and under Maintenance multiple times within an hour. Static data management systems, such as fixed-size arrays, are fundamentally ill-suited for this dynamic environment because they require expensive $O(n)$ shifting operations every time a station is added or removed.

This project implements a Smart EV Charging Network — a terminal-based C++ application that dynamically manages charging stations using a Singly Linked List data structure. The system demonstrates how the linked list's $O(1)$ pointer rewiring enables efficient real-time updates to a live network of stations, without disrupting the overall infrastructure or requiring dataset restructuring.

The project is part of the Data Structures and Algorithms curriculum at ITM Skills University and serves as a practical demonstration of how foundational computer science concepts map directly to real-world infrastructure challenges facing smart cities today.

2. Problem Statement / Case Background (Abstract)

Problem Statement

Design and implement a command-line Smart EV Charging Network in C++ that allows a network operator to:

- Dynamically add new charging stations to the network as they become operational
- Remove stations from the active network when they enter maintenance mode
- Update the real-time status of any station (Available / Occupied / Maintenance)
- Display the full network dashboard with color-coded status indicators
- Search for a specific station by ID with visual highlighting in the dashboard
- Manage a waiting vehicle queue per station when all slots are occupied

Background

Large EV charging networks such as ChargePoint, Tata Power EV, and EESL India manage thousands of distributed charging stations across metropolitan areas. These networks face three core operational challenges that a static data structure cannot efficiently address:

Challenge	Impact	Linked List Solution
Frequent station additions	New stations installed weekly as city EV adoption grows	$O(1)$ pointer append at tail
Unplanned maintenance removals	Stations go offline unexpectedly — need instant removal	$O(n)$ traversal + $O(1)$ pointer rewire
Real-time status updates	Status changes every few minutes as vehicles plug in/out	$O(n)$ traversal + in-place field update

The linked list's memory model — where each node is an independently allocated object connected via pointer — directly mirrors the physical reality of a charging network, where each station is an independent unit that can be added, removed, or updated without affecting its neighbors.

3. System Design & Architecture

3.1 System Architecture

The system is organized into four source files following standard C++ separation of interface and implementation:

File	Lines	Purpose
station.h	~18	ChargingStation struct — node definition with constructor
network.h	~22	ChargingNetwork class — method declarations, private head pointer
network.cpp	~160	Full method implementations: insert, delete, search, update, display
ui.h	~60	ANSI styling constants, terminal helpers: padRight, padLeft, center, trim, statusColor
main.cpp	~90	Menu loop, user input handling, clearScreen, banner, dispatch

3.2 Node Structure

Each charging station in the network is represented as a node in the singly linked list. The ChargingStation struct contains all relevant operational data:

```
struct ChargingStation {
    string id;           // Unique station identifier (e.g., ST001)
    string location;     // Physical location (e.g., Kharghar Sec 5)
    int    capacity;     // Charging power in kilowatts (kW)
    string status;       // "Available" | "Occupied" | "Maintenance"
    int    waitingCount; // Number of vehicles queued at this station
    ChargingStation* next; // Pointer to the next node in the list
};
```

3.3 Linked List Architecture

The ChargingNetwork class encapsulates a private head pointer and exposes all network operations as public methods. The linked list grows and shrinks dynamically as stations are added and removed — memory is allocated with new on insert and freed with delete on removal:

Network state at any given time:

```
[head] → [ST001|Kharghar|50kW|Available|0] → [ST002|Vashi|100kW|Occupied|2] →  
[ST003|Belapur|75kW|Available|0] → NULL
```

If ST002 enters maintenance, the pointer from ST001 is rewired to ST003, and ST002's memory is freed:

```
[head] → [ST001|Kharghar|50kW|Available|0] → [ST003|Belapur|75kW|Available|0] →  
NULL
```

3.4 Data Flow

The application follows a clean layered architecture. All user interaction is handled in main.cpp through a continuous menu loop. The loop dispatches to ChargingNetwork methods, which traverse and modify the linked list in-place. Results are returned as ANSI-styled strings and rendered to the terminal. The displayAll() method performs a full traversal on every refresh, printing a live dashboard with aligned columns, status colors, and a footer summary.

Layer	Component	Responsibility
Presentation	main.cpp + ui.h	Menu rendering, user input, ANSI terminal styling
Logic	network.cpp	Linked list traversal, insertion, deletion, update
Data	station.h (nodes)	Node struct with operational fields and next pointer

4. Methods, Algorithms & Technology Applied

4.1 Core Linked List Operations

The system implements six core operations on the linked list, each with a defined time complexity:

Operation	Method	DS Concept	Time Complexity	Real-World Meaning
Add Station	addStation()	Insert at tail	O(n)	New station installed in city
Remove Station	removeStation()	Pointer rewire + free	O(n)	Station enters maintenance
Update Status	updateStatus()	Traverse + in-place edit	O(n)	Vehicle plugs in or out
Display All	displayAll()	Full traversal	O(n)	Network dashboard refresh
Search Station	searchStation()	Linear search by ID	O(n)	Driver looks up a station
Queue Vehicle	incrementWaiting()	Traverse + counter++	O(n)	Vehicle joins waiting queue

4.2 Insertion Algorithm

New stations are inserted at the tail of the list. The method first checks for duplicate IDs using the private exists() helper (O(n) traversal), then traverses to the last node and appends the new node:

```
if (!head) { head = node; }           // empty list: set as head
else {
    ChargingStation* cur = head;
    while (cur->next) cur = cur->next; // traverse to tail
    cur->next = node;                 // append
}
```

4.3 Deletion Algorithm

Deletion handles two cases: removing the head node and removing a mid-list or tail node. In both cases, the deleted node's memory is explicitly freed using delete to prevent memory leaks:

```
if (head->id == id) {                 // Case 1: delete head
    ChargingStation* del = head;
```

```

    head = head->next;
    delete del;
}
// Case 2: traverse with prev pointer
prev->next = cur->next;          // rewire: skip deleted node
delete cur;

```

4.4 Status Canonicalization

User input for status updates is normalized through a `canonicalStatus()` function that converts any case variation ("OCCUPIED", "occupied", "Occupied") to the canonical form used internally. This prevents status mismatches when filtering or displaying stations.

4.5 Terminal UI & ANSI Styling

The `ui.h` header defines a complete terminal styling system using ANSI escape codes with 256-color palette support. Status indicators use color-coded dot prefixes: green for Available, red for Occupied, and yellow for Maintenance. The `displayAll()` method renders a Unicode box-drawing table with aligned columns, a header row, a summary footer, and optional row highlighting for search results.

Technology	Purpose
C++17	Core programming language
g++ Compiler	Compilation: <code>g++ -o ev main.cpp network.cpp</code>
ANSI Escape Codes	Terminal colors (256-color palette) and cursor control
Unicode Box Drawing	┌┐└┘┌┐└┘┌┐└┘┌┐└┘ for dashboard table borders
STL string / getline	Input handling with whitespace trimming via <code>trim()</code>
new / delete	Manual heap memory management for linked list nodes
Git (v1.0.0 → v1.1.3)	Version control with semantic tags across development

5. Implementation Details

5.1 File Structure

File	Lines	Description
main.cpp	~90	Application entry point — menu loop, banner, input dispatch, clearScreen
network.h	~22	ChargingNetwork class declaration — private head, public method signatures
network.cpp	~160	All method implementations, displayAll() with full Unicode table rendering
station.h	~18	ChargingStation struct — fields and constructor with default initialization
ui.h	~60	ANSI color constants, screen control, padRight/padLeft/center/trim helpers

5.2 Compilation

The project is compiled using the following command:

```
g++ -std=c++17 -o ev main.cpp network.cpp
```

station.h and ui.h are header-only and included where needed. The compiled binary ev is executed directly from the terminal. Three version binaries are preserved in the repository: ev_v1.0.0, ev_v1.1.0, and ev_v1.1.3.

5.3 Version History

Version	Changes
v1.0.0	Initial implementation — core linked list with add, remove, update, display, search
v1.1.0	Added waiting vehicle queue (incrementWaiting), improved menu layout
v1.1.3 (current)	Polished ANSI styling, statusColor helper, canonical status normalization, search highlight

5.4 Application Walkthrough

Below is a representative terminal session demonstrating the core operations:

Step 1 — System starts with an empty network. The dashboard shows a placeholder message.

Step 2 — Operator adds ST001 (Kharghar Sec 5, 50 kW). Node inserted at head (list was empty).

Step 3 — ST002 (Vashi Plaza, 100 kW) added. Node appended at tail via traversal.

Step 4 — ST003 (Belapur Station, 75 kW) added. Three-node list now active.

Step 5 — Vehicle plugs in at ST001. Status updated: Available → Occupied (green dot turns red).

Step 6 — Second vehicle arrives at ST001 (already Occupied). Queue incremented: Waiting: 1.

Step 7 — ST002 searched by ID. Row highlighted with  found indicator in dashboard.

Step 8 — ST002 removed for maintenance. Pointer from ST001 rewired to ST003. Memory freed.

Step 9 — Dashboard shows 2 stations: 1 Occupied, 1 Available. Footer summary updated automatically.

6. Results and Conclusion

6.1 Results

The Smart EV Charging Network was successfully implemented with the following capabilities:

- Dynamic station addition — new nodes inserted at tail with duplicate ID validation
- Station removal — head and mid-list deletion handled with correct pointer rewiring and memory deallocation
- Real-time status updates — case-insensitive input normalized to canonical status values
- Live network dashboard — Unicode table with ANSI color-coded status dots and summary footer
- Station search with visual highlight — matching row marked with  found indicator
- Waiting vehicle queue — per-station counter incremented independently of status
- Clean memory management — all nodes freed on exit via destructor freeAll()
- Version-controlled development — three git tags from v1.0.0 to v1.1.3 with bug tracking

6.2 DSA Concepts Demonstrated

Concept	Implementation
Singly Linked List	ChargingStation nodes connected via next pointer, traversed from head
Dynamic Memory Allocation	new for node creation, delete for node removal, destructor for cleanup
Linear Search	O(n) traversal to find node by ID in search, update, delete, and queue operations
Pointer Rewiring	prev->next = cur->next to remove a node without shifting data
Head Node Special Case	Separate branch in removeStation() when deleting the head node
Encapsulation	ChargingNetwork class hides head pointer and internal helpers as private members
Separation of Concerns	station.h (data), network.h/cpp (logic), ui.h (presentation), main.cpp (control)

6.3 Performance Analysis

Operation	Time	Space	Notes
addStation()	O(n)	O(1)	O(n) to reach tail; O(1) for the append itself
removeStation()	O(n)	O(1)	O(n) traversal; O(1) pointer rewire

Operation	Time	Space	Notes
updateStatus()	$O(n)$	$O(1)$	Linear scan; in-place field modification
searchStation()	$O(n)$	$O(1)$	Linear scan; no data moved
displayAll()	$O(n)$	$O(1)$	Single pass; prints and counts in same traversal
Overall Space	—	$O(n)$	n nodes, each allocated independently on heap

6.4 Conclusion

This project demonstrates the practical application of Singly Linked Lists in managing dynamic, real-world infrastructure. By enabling $O(1)$ pointer-based insertions and deletions — without the $O(n)$ shifting cost of arrays — the linked list proves ideal for environments where the dataset changes frequently and unpredictably.

The terminal-based EV charging dashboard illustrates that foundational data structures are not merely academic exercises. They are the same mechanisms powering real charging network management systems, IoT device registries, and urban mobility platforms. The project additionally demonstrates industry-standard C++ practices: manual memory management, header-only utility modules, ANSI terminal UI, and semantic version control with Git.

Future extensions planned for this project include a simulation mode (V2) that auto-generates 80+ stations and spawns randomized vehicles to benchmark $O(n)$ search performance at scale, and a web interface (V3) using FastAPI and Leaflet.js with real OpenStreetMap coordinates for live station visualization across Navi Mumbai.

7. References

GeeksForGeeks — Linked List Data Structure

<https://www.geeksforgeeks.org/data-structures/linked-list/>

GeeksForGeeks — Dynamic Memory Allocation in C++

<https://www.geeksforgeeks.org/new-and-delete-operators-in-cpp-for-dynamic-memory/>

cppreference.com — std::string and ANSI Escape Codes

https://en.cppreference.com/w/cpp/string/basic_string

ITM Skills University — DSA Problem Statement: Case Study 182, Smart EV Charging Network

Project Repository — Smart EV Charging Network (AASHPAW)

https://github.com/AASHPAW1015/EV_spr2proj

Claude Sonnet 4.6 (medium) — Used for ideation, debugging assistance, and report drafting